

Análise Comparativa de Desempenho: Insertion Sort e Bucket Sort em C++ com e sem Paralelismo.

André H. S. Silva¹, Guilherme A. D. Alves², Nathan L. S. Oliboni³

¹ Centro de Ciências Exatas e da Terra – Universidade Estadual do Oeste do Paraná (UNIOESTE)

Caixa Postal 711 – 85.819-110 – Cascavel – PR – Brazil

{andre.silva71, guilherme.alves6, nathan.oliboni}@unioeste.br

Abstract. *This meta-paper presents a performance comparison between two sorting algorithms, Insertion Sort and Bucket Sort, both implemented in C++. The evaluation considers different input sizes, data types (random, sorted, partially sorted, and reverse sorted), and the impact of parallelism. As a complementary analysis, it also includes a brief performance comparison between Python and C++ implementations of the sorting methods.*

Resumo. *Este meta-artigo aborda uma comparação de desempenho de dois algoritmos de ordenação, Insertion Sort e Bucket Sort, ambos com uma implementação realizada em C++, considerando diferentes tamanhos de entrada, tipos de dados (aleatórios, ordenados, parcialmente ordenados e decrescentes) e o impacto do paralelismo. Como complemento, também apresenta-se uma breve comparação de desempenho entre Python e C++ nos métodos de ordenação*

1. Introdução

O propósito fundamental da ciência da computação não é apenas desenvolver algoritmos que resolvam problemas, mas criar soluções que o façam de forma eficiente e a análise de algoritmos permite avaliar a eficiência de diferentes soluções para um mesmo problema. Este trabalho foca na comparação de dois algoritmos de ordenação: o *Insertion Sort* e o *Bucket Sort*.

A ordenação por *Insertion Sort* é um algoritmo de ordenação simples que funciona inserindo iterativamente cada elemento de uma lista não ordenada em sua posição correta em uma parte ordenada da lista. É como ordenar cartas de baralho em suas mãos. Você divide as cartas em dois grupos: as ordenadas e as não ordenadas. Em seguida, você escolhe uma carta do grupo não ordenado e a coloca no lugar correto no grupo ordenado[1].

1. Começa no segundo elemento do *array*, pois o primeiro elemento é considerado classificado[1].
2. Compara-se o segundo elemento com o primeiro elemento. Se o segundo elemento for menor, os troca[1].
3. Mova o terceiro elemento, compara com os dois primeiros elementos e o coloca na posição correta[1].
4. Repete até que toda a matriz esteja ordenada[1].

O *Bucket Sort* é uma técnica de ordenação que envolve a divisão de elementos em vários grupos, ou baldes. Esses baldes são formados pela distribuição uniforme dos elementos. Uma vez que os elementos são divididos em *buckets*, eles podem ser

ordenados usando o algoritmo *Insertion Sort*, por exemplo. Finalmente, os elementos são reunidos de forma ordenada[2].

A eficiência será quantificada utilizando a notação *Big O*, que é um conceito matemático normalmente empregado para medir o custo de um algoritmo. No geral, significa encontrar uma função que classifica os algoritmos de acordo com seu tempo de execução conforme o tamanho da entrada (n) de dados aumenta. Comumente a função que descreve a performance de um algoritmo se refere ao seu limite superior, que determina o pior caso (de entrada de dados) do algoritmo[3].

É possível calcular que a complexidade do algoritmo do *Insertion Sort* no melhor caso, ou seja, quando a lista já estiver ordenada é $O(n)$, onde n é o número de elementos na lista, entando no pior caso, quando a lista estiver em ordem reversa, é $O(n^2)$ [1].

Já no algoritmo do *Bucket Sort* o melhor caso, quando cada balde recebe um número igual de elementos, é representado por $O(n + k)$, sendo k o número de baldes. Enquanto no pior caso, quando todos os elementos acabam em um único balde, então se executa o *Insertion Sort* de complexidade quadrática em n , é $O(n^2)$ [2].

Adicionalmente, este estudo explora o paralelismo, uma técnica de programação que divide uma grande quantidade de dados em partes menores que podem ser operadas em paralelo. Após os dados serem processados, eles são combinados novamente em um único conjunto. Particionando os dados, pode-se utilizar toda a força de processamento disponível[4].

2. Objetivos

O trabalho tem como objetivo geral comparar o comportamento, em termos de tempo de execução, do algoritmo *Insertion Sort* tradicional frente a uma abordagem híbrida que o combina com o *Bucket Sort*, analisando o desempenho em diferentes cenários e condições de execução. Para atingir este objetivo, foram definidos os seguintes objetivos específicos:

- Implementar e comparar o desempenho cronológico do *Insertion Sort* tradicional com a abordagem de *Bucket Sort* que utiliza o próprio *Insertion Sort* para a ordenação interna dos baldes.

- Analisar o impacto do número de baldes (10, 100 e 1000) no tempo de execução da abordagem híbrida, a fim de determinar uma configuração ótima para os conjuntos de dados testados.
- Avaliar e comparar a performance das implementações em duas linguagens com paradigmas distintos: C++ e Python.
- Investigar o ganho de desempenho proporcionado pelo paralelismo, comparando as execuções em modo *single-thread* contra as execuções em *multi-thread* para ambas as estratégias de ordenação e linguagens.
- Analisar o comportamento dos algoritmos frente a quatro tipos de entrada de dados distintos — aleatórios, ordenados, decrescentes e parcialmente ordenados — para corroborar como a distribuição inicial dos elementos afeta a eficiência de cada método.

3. Materiais e Métodos

As ferramentas empregadas para o desenvolvimento dos algoritmos foram as linguagens de programação Python e C++. Ambas são linguagens de alto nível, sendo Python interpretada e C++ compilada. Para a manipulação de múltiplas *threads*, foram utilizadas as bibliotecas ‘*threading*’ (Python) e ‘*thread*’ (C++). Em ambos os casos, o editor de texto Visual Studio Code foi empregado.

Para a execução dos testes, utilizou-se um notebook Acer Nitro 5 com as seguintes configurações:

- Processador: AMD Ryzen 7 4800H
 - Frequência de clock: 2.9Ghz à 4.2Ghz
 - Cores: 8
 - *Threads*: 16
 - L1 *Instruction* Cache: 8 x 32 KB
 - L1 *Data* Cache: 8 x 32 KB
 - L2 Cache: 8 x 512 KB
 - L3 Cache: 8 MB
- Placa de vídeo: GTX 1650ti 4Gb Vram
 - Não utilizada nos testes
- Armazenamento: SSd 1Tb Kingston M.2
- Memória: 16Gb
- Sistema Operacional: Ubuntu 22.04 Lts

Foram utilizados quatro tipos de conjuntos de dados: aleatórios, ordenados, decrescentes e parcialmente ordenados, com tamanhos que variaram de 100 a 2.000.000 de elementos. Para cada configuração, foram executados testes de ambos os métodos de ordenação (*Insertion Sort* e *Bucket Sort* com *Insertion Sort*) com e sem o uso de *multithreading* (utilizando 16 *threads*). No caso específico do *Bucket Sort*, os testes foram repetidos para diferentes quantidades de buckets (10, 100 e 1000). Para garantir a confiabilidade estatística, cada cenário de teste foi executado seis vezes, sendo a primeira descartada.

3.1. Implementação em Python

A estratégia de paralelização para o Insertion Sort consistiu em dividir a lista de entrada em sublistas, onde cada *thread* era responsável por ordenar uma partição de forma independente. No caso do Bucket Sort, a paralelização ocorreu na etapa de ordenação dos "baldes": após a distribuição sequencial dos elementos, os baldes foram distribuídos entre as *threads* para serem ordenados simultaneamente.

Os dados de desempenho, como o tempo de execução dos métodos, foram medidos e sistematicamente registrados em arquivos CSV para posterior análise.

3.2. Implementação em C++

A implementação do Insertion Sort segue o algoritmo clássico de ordenação por inserção, onde cada elemento da lista é inserido na posição correta em relação aos elementos já ordenados. O algoritmo percorre a lista a partir do segundo elemento, comparando-o com os elementos anteriores e deslocando-os quando necessário até encontrar a posição adequada para inserção.

A estratégia de paralelização para o Insertion Sort consistiu em dividir a lista de entrada em chunks (sublistas) proporcionais ao número de *threads* disponíveis, onde cada *thread* era responsável por ordenar uma partição de forma independente usando o algoritmo de insertion sort. Após a conclusão da ordenação paralela dos chunks, foi implementado um processo de ‘merge’ hierárquico que combina os segmentos ordenados de forma iterativa, dobrando o tamanho dos ‘gaps’ a cada iteração até que toda a lista esteja completamente ordenada.

O Bucket Sort foi implementado seguindo a estratégia de distribuição dos elementos em baldes baseada no range de valores. O algoritmo calcula o valor mínimo e máximo da lista, determina o range total e distribui os elementos proporcionalmente entre os buckets. Cada bucket é então ordenado individualmente usando o insertion sort, e posteriormente todos os buckets são concatenados para formar a lista final ordenada.

No caso do Bucket Sort, a paralelização ocorreu na etapa de ordenação dos "baldes": após a distribuição sequencial dos elementos pelos *buckets*, os baldes foram distribuídos entre as *threads* para serem ordenados simultaneamente. O algoritmo limita o número de *threads* ao mínimo entre o número de *threads* especificado e o número de *buckets* disponíveis, calculando quantos *buckets* cada *thread* deve processar. Cada *thread* executa o insertion sort em seu conjunto de buckets designado de forma independente, maximizando a eficiência do processamento paralelo.

Os dados de desempenho, como o tempo de execução dos métodos, foram medidos utilizando a biblioteca ‘*ctime*’ e sistematicamente registrados em arquivos CSV para posterior análise.

3.3. Análise

Para a análise dos dados, foram empregadas as bibliotecas Pandas e Matplotlib. Pandas, uma biblioteca open-source para Python, foi utilizada para a manipulação e

análise dos dados. Sua principal funcionalidade, o *DataFrame*¹, permite a organização, limpeza, transformação e análise eficiente de grandes volumes de dados, sendo responsável pela leitura dos arquivos CSV contendo as métricas e pela facilitação de sua manipulação. Matplotlib, por sua vez, é uma biblioteca abrangente para a criação de visualizações estáticas, animadas e interativas em Python, e foi empregada para a geração dos gráficos de resultados.

Foram analisadas informações como algoritmo de ordenação, tempo de execução, tamanho da lista, uso de paralelismo e número de buckets.

4. Resultados

4.1 Teórica

Com uma análise assintótica, que descreve o comportamento do algoritmo para grandes entradas, chegou-se ao polinômio de custo assintótico $5n^2 + 6n$ para o pior caso da solução Insertion Sort em C++ enquanto no melhor caso encontrou-se $13n + 7$ e para a solução Bucket Sort na mesma linguagem $5n^2 + 19n + 6k + 24$ como polinômio de pior caso e $26n - 5k + 43$ como melhor caso, com um foco no termo de maior grau, ignorando constantes e termos menores, conclui-se que a complexidade no pior caso em tal cenário é $O(n^2)$. Na análise dos algoritmos em Python chegou-se no polinômio $5n^2 + 8n - 3$ para o pior caso do Insertion Sort e $14n - 1$ para o melhor caso, enquanto para o Bucket Sort encontra-se $5n^2 + 20n + 6k + 23$ para o pior caso e $28n + 5k + 17$ para o melhor caso, mantendo a complexidade $O(n^2)$ com o pior cenário.

4.2 Prática

```
1 void insertionSortMultithread(vector<int>& lista, int numThreads, int n, const string&
  tipoEntrada, int execucao) {
2     int size = lista.size();
3     int chunkSize = size / numThreads;
4     vector<thread> threads;
5     for(int i = 0; i < numThreads; i++) {
6         int start = i * chunkSize;
7         int end = (i == numThreads - 1) ? size : (i + 1) * chunkSize;
8         threads.push_back(thread(insertionSortRange, ref(lista), start, end));
9     }
10    for(auto& t : threads) {
11        t.join();
12    }
13    for(int gap = chunkSize; gap < size; gap *= 2) {
14        for(int i = 0; i < size; i += gap * 2) {
15            int mid = min(i + gap, size);
16            int end = min(i + gap * 2, size);
17            if(mid < end) {
18                mergeRanges(lista, i, mid, end);
19            }
20        }
21    }
22 }
```

¹ Uma estrutura de dados tabular bidimensional, semelhante a uma planilha ou tabela, onde os dados são organizados em linhas e colunas.

Figura 1. Código da implementação *Multi Thread* do Insertion Sort.

Na figura 1 é apresentada a implementação do algoritmo de ordenação Insertion Sort utilizando a estratégia do *multithreading*. Seu funcionamento se dá dividindo o trabalho entre múltiplas threads da seguinte forma:

1. Vetor original é dividido em *chunks* (pedaços) iguais, onde cada *thread* é responsável por ordenar uma parte do vetor.
2. Cada *thread* executa o Insertion Sort tradicional em seu *chunk*.
3. O programa aguarda todas as *threads* completarem sua ordenação antes de prosseguir.
4. Após a ordenação, o algoritmo realiza o *merge* (junção) para combinar os pedaços ordenados em um único vetor totalmente ordenado.

O processo de *merge* acontece em níveis, começando com *chunks* menores progressivamente combinando-os em grupos maiores, até todo o vetor esteja ordenado. Ele realiza essa tarefa da seguinte forma:

1. Comparação de elementos do primeiro segmento com os do segundo segmento, escolhendo o menor para colocar no vetor temporário.
2. Quando um segmento é completamente processado, os elementos restantes do outro segmento são copiados diretamente.
3. Por fim, todos os elementos do vetor temporário são copiados de volta para suas posições no vetor original.

```
1 void bucketSortMultithread(vector<int>& lista, int numBuckets, int numThreads, int n,
2   const string& tipoEntrada, int execucao) {
3     vector<vector<int>> buckets(numBuckets);
4     separarBuckets(lista, buckets, numBuckets);
5     vector<thread> threads;
6     int threadsToUse = min(numThreads, numBuckets);
7     int bucketsPerThread = numBuckets / threadsToUse;
8     for(int t = 0; t < threadsToUse; t++) {
9         int startBucket = t * bucketsPerThread;
10        int endBucket = (t == threadsToUse - 1) ? numBuckets : (t + 1) * bucketsPerThread;
11
12        threads.push_back(thread([&buckets, startBucket, endBucket, n, tipoEntrada]() {
13            for(int i = startBucket; i < endBucket; i++) {
14                insertionSortBucket(buckets[i]);
15            }
16        }));
17    }
18    for(auto& t : threads) {
19        t.join();
20    }
21    lista.clear();
22    for(const auto& bucket : buckets) {
23        lista.insert(lista.end(), bucket.begin(), bucket.end());
24    }
25 }
```

Figura 2. Código da implementação *Multi Thread* do Bucket Sort.

Na figura 2 é apresentada a implementação do algoritmo do Bucket Sort utilizando a estratégia do *multithreading*. Seu funcionamento se dá da seguinte forma:

1. Primeiro, todos os elementos do vetor são distribuídos em baldes com base em seus valores.
2. O algoritmo então divide os baldes entre as *threads* disponíveis de forma equilibrada, onde cada *thread* fica responsável por ordenar um conjunto específico.
3. Cada *thread* executa o Insertion Sort nos baldes que foram atribuídos a ela.
4. O programa aguarda que todas as *threads* finalizem sua ordenação antes de prosseguir.
5. Após a ordenação, todos os baldes são concatenados para formar o vetor final ordenado.

A separação dos elementos nos baldes é realizado pela função “**separarBuckets**”, presente na linha 3 da Figura 2. O processo de distribuição é realizado da seguinte forma:

1. A função identifica o valor máximo e mínimo do vetor para determinar o intervalo total dos valores.
2. Calcula o tamanho ideal de cada *bucket*, dividindo o *range* total pelo número de *buckets* desejados.
3. Cada elemento é colocado no *bucket* correto considerando sua posição relativa no *range* de valores.
4. Há o tratamento para casos como: quando há mais *buckets* que valores possíveis, o número de *buckets* é ajustado automaticamente para um número efetivo.

4.2.1 Insertion Sort - Single Thread e Multi Thread em C++

Nas Figuras 3 e 4 são mostrados os gráficos de tempo de execução pelo tamanho da entrada com base no tipo de entrada (aleatórios, decrescentes, parcialmente ordenados e ordenados). Na primeira imagem percebe-se a execução de maneira *Single Thread*, que apresenta linhas baseadas no comportamento assintótico esperado do algoritmo, onde no pior caso, número ordenados do maior para o menor, ele apresenta o maior tempo de execução, com um comportamento $O(n^2)$, enquanto para o conjunto ordenado, melhor dos casos, ele apresenta um comportamento $O(n)$. Comparando ambas as imagens, na segunda o comportamento e variação conforme o tipo de entrada se mantém, porém com uma diferença crucial, a redução do tempo de execução, que utilizando *Multi Thread* caiu para um pico de aproximadamente 1400 segundos, uma redução de cerca de 86% de tempo, o que mostra que utilizar a paralelização para o método do Insertion Sort pode ser uma estratégia eficaz nesse cenário.

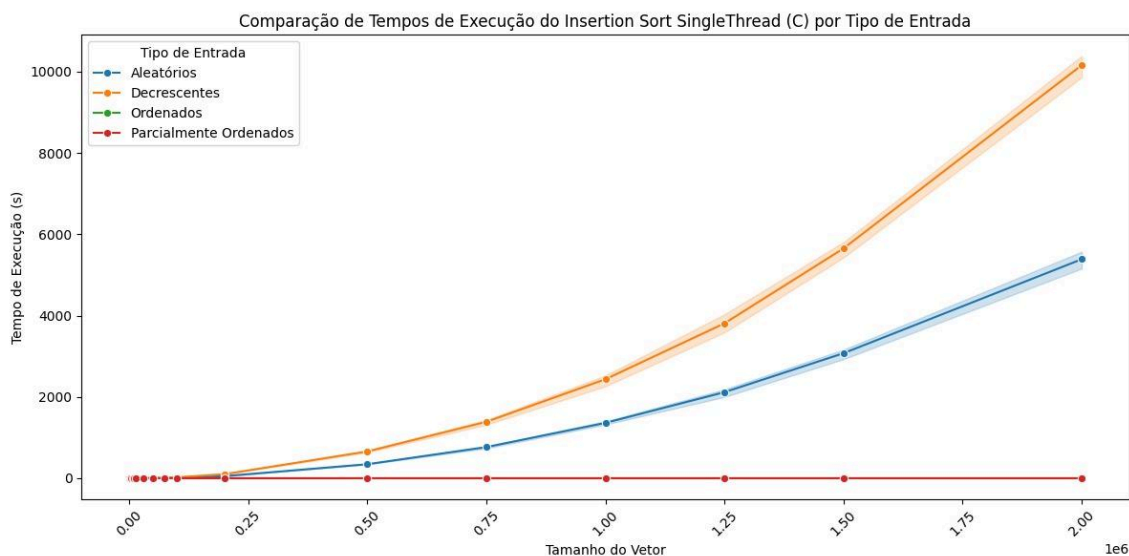


Figura 3. Insertion Sort *Single Thread* x Tamanho de entrada x Tipo de entrada

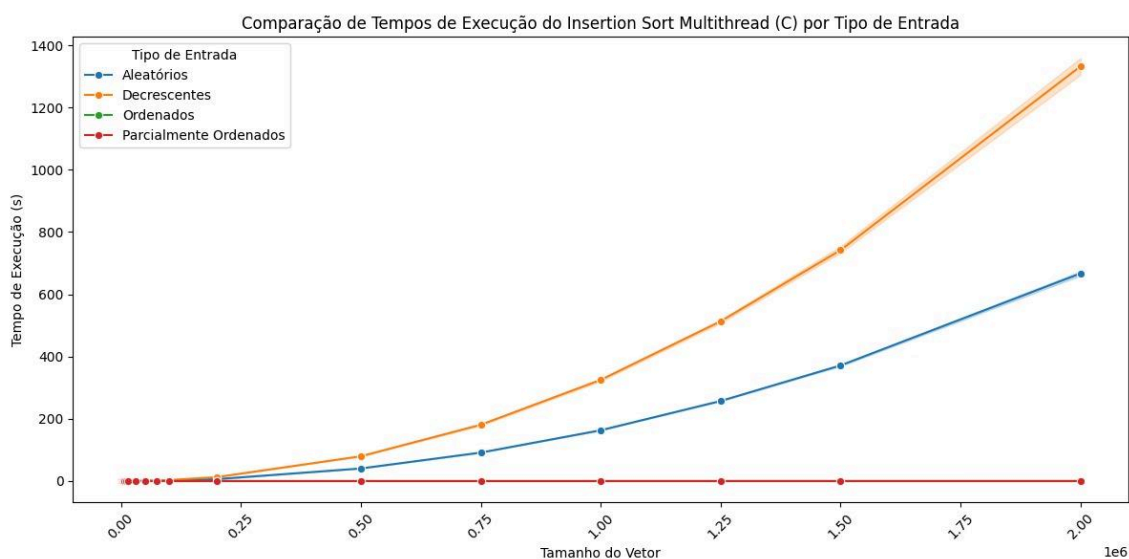


Figura 4. Insertion Sort *Multi Thread* x Tamanho de entrada x Tipo de entrada

Nas Figuras 5, 6, 7 e 8 são apresentados os gráficos com a média das 5 execuções contabilizadas para os conjuntos: aleatórios, parcialmente ordenados, ordenados e decrescentes respectivamente. Ao fazer uma análise de cada um deles, é visível que seus comportamentos refletem aos gráficos das Figuras 3 e 4, também crescendo conforme o tamanho de elementos da entrada aumenta, e revelando ainda mais a eficácia que a paralelização do método traz para cada um dos conjuntos, a exemplo do pior caso, que serialmente chegou a um pico de aproximadamente 10000 segundos para 2 milhões de elementos, enquanto paralelizado chegou a menos de 2000 segundos, uma redução de 80% de tempo de execução.

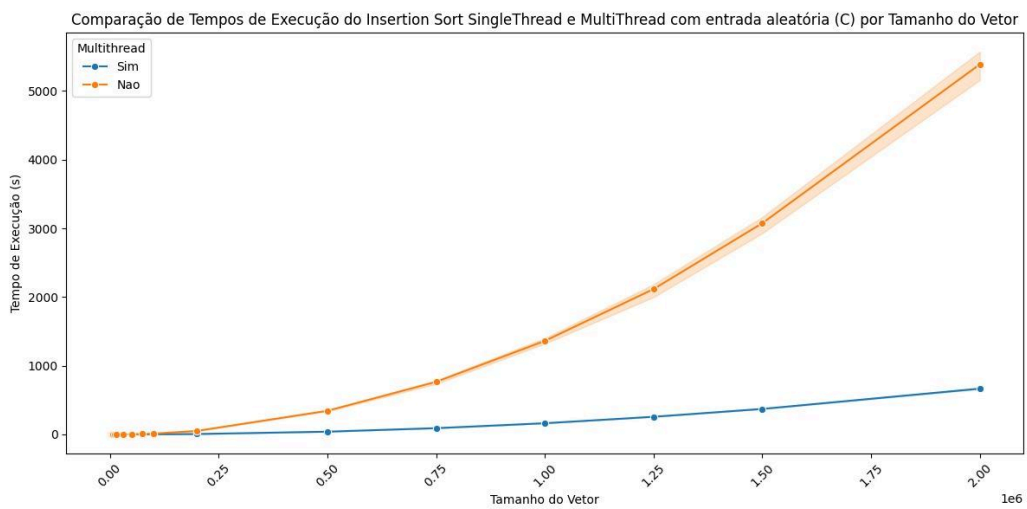


Figura 5. Insertion Sort *Multi Thread* x *Single Thread* x Número Aleatórios

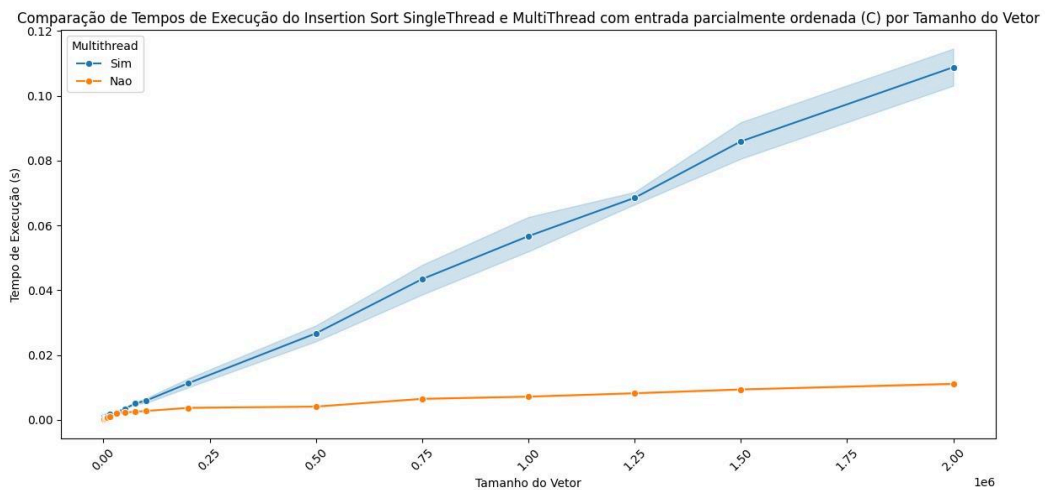


Figura 6. Insertion Sort *Multi Thread* x *Single thread* X Números parcialmente ordenados

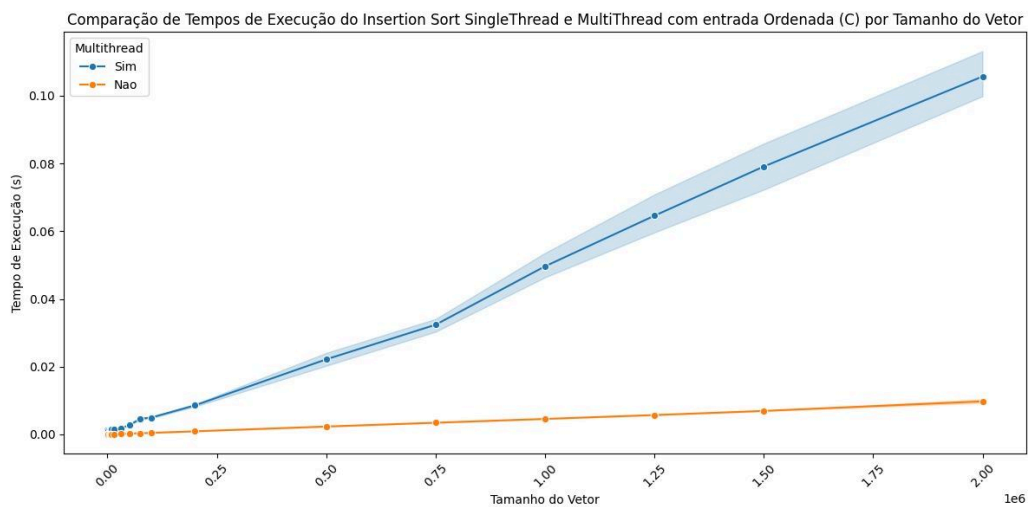


Figura 7. Insertion Sort *Multi Thread* x *Single Thread* x Números Ordenados

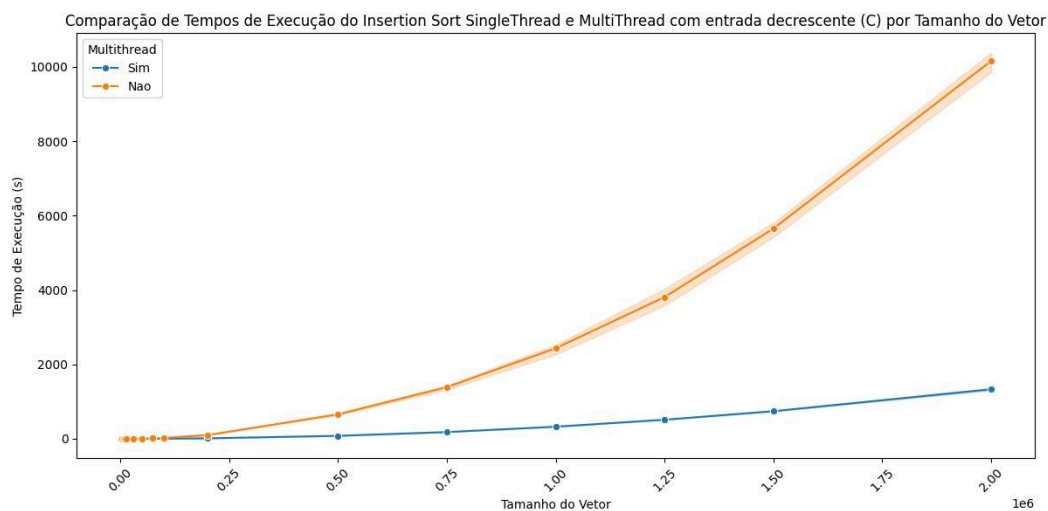


Figura 8. Insertion Sort *Multi Thread* x *Single Thread* x Números Decrescentes

O Quadro 1 detalha o tempo médio de execução do algoritmo Insertion Sort para diferentes tipos de entrada de dados (Aleatórios, Decrescentes, Ordenados e Parcialmente Ordenados), tanto em sua versão *single thread* (Não) quanto *multi thread* (Sim).

Observa-se uma clara distinção no desempenho do Insertion Sort entre as execuções *single thread* e *multi thread*. Na configuração *single-thread*, o tempo médio de execução é significativamente maior, especialmente para entradas "Decrescentes" (1212.404 segundos) e "Aleatórios" (656.183 segundos), que representam cenários mais desafiadores para este algoritmo. Em contrapartida, para entradas "Ordenados" e "Parcialmente Ordenados" em *single-thread*, o tempo de execução é negligenciável (0.002 segundos), o que é esperado, dado que o Insertion Sort apresenta sua melhor performance com listas já ordenadas ou quase ordenadas.

Ao analisar a versão *multi thread*, percebe-se uma redução drástica nos tempos médios de execução em todas as categorias de entrada. Para dados "Decrescentes", o

tempo cai de 1212.404 s para 159.579 s, e para "Aleatórios", de 656.183 s para 80.018 s. Embora as entradas "Ordenados" e "Parcialmente Ordenados" também exibam tempos ligeiramente maiores em *multi-thread* (0.018 s e 0.022 s, respectivamente) comparados ao *single-thread*, essa diferença é irrisória. A redução percentual mais expressiva ocorre nos cenários de pior caso e caso médio, onde o paralelismo demonstra sua eficácia em acelerar o processo de ordenação do Insertion Sort.

Multithread	Tipo de Entrada	Tempo de Execução Médio (s)
Nao	Aleatórios	656.183
Nao	Decrescentes	1212.404
Nao	Ordenados	0.002
Nao	Parcialmente Ordenados	0.002
Sim	Aleatórios	80.018
Sim	Decrescentes	159.579
Sim	Ordenados	0.018
Sim	Parcialmente Ordenados	0.022

Quadro 1. Comparação entre os tipos de entrada para o Insertion Sort, *multi thread* e *single thread*.

4.2.2 Bucket Sort - *Single Thread* e *Multi Thread* em C++

Nas Figuras 9 e 10 são mostrados os gráficos de tempo de execução pelo tamanho de entrada com base no número de baldes utilizados pelo método. Na sétima imagem nota-se a média com base nas execuções gerais em modo *Single Thread*, e um comportamento esperado acontece, conforme o número de elementos aumenta, quanto maior for o número de baldes, menor será o tempo, devido ao comportamento do caso médio, onde os baldes contém a mesma quantidade de elementos $O(n + k)$. Na imagem 10, indo contra o comportamento esperado e visto no Insertion Sort anteriormente, o uso do paralelismo acabou não se mostrando tão efetivo, uma vez que esperava-se que o tempo de execução diminuísse, porém em diversos momentos ele foi igual, próximo ou até mesmo superior ao método de maneira *Single Thread*. O fato do paralelismo não ser tão efetivo pode ter ocorrido devido ao overhead de sincronização que pode ter

acontecido durante o momento da sincronização das *threads* e dos baldes que elas estavam utilizando, e isso se mostrou visível na Figura 11.

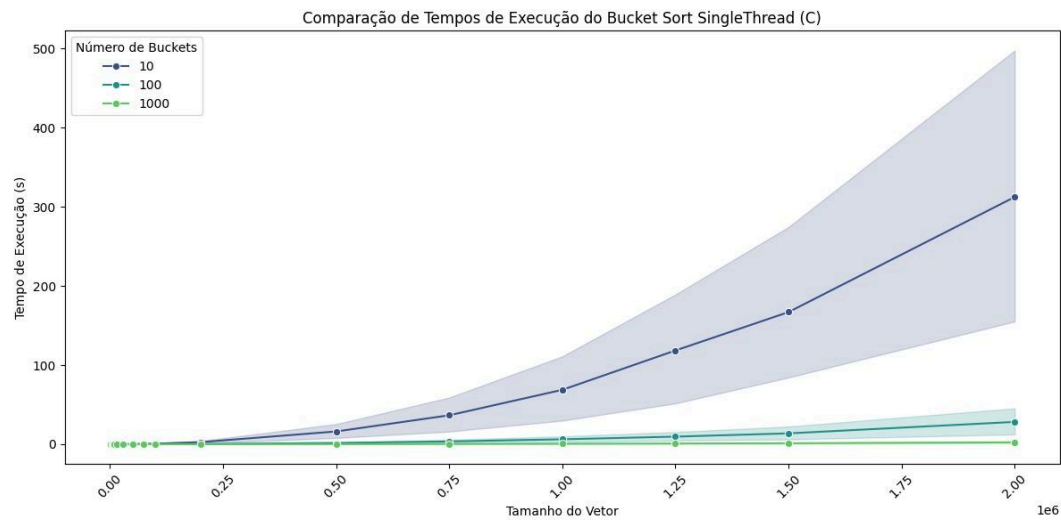


Figura 9. Bucket Sort *Single Thread* x Número de baldes x Tamanho da Entrada.

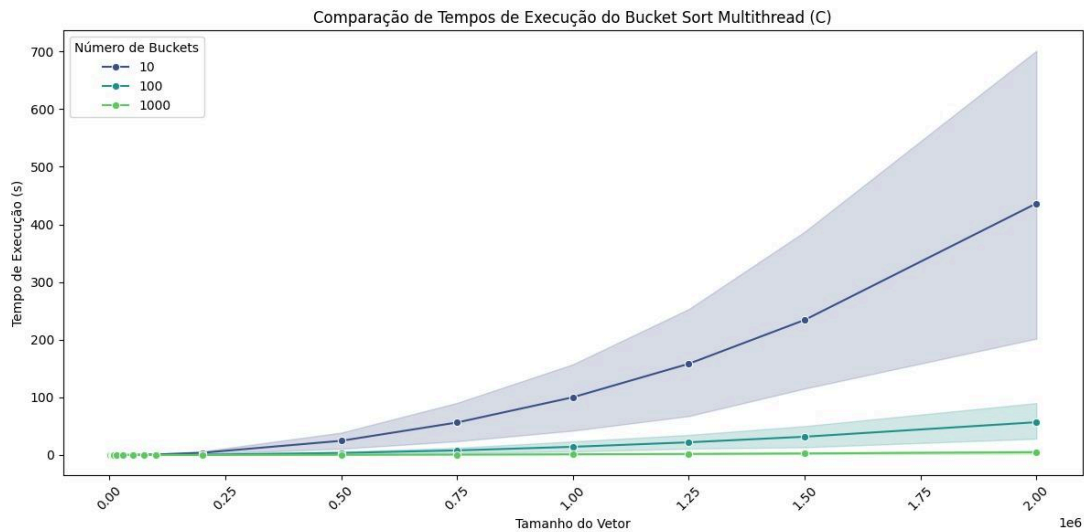


Figura 10. Bucket Sort *Multi Thread* x Número de baldes x Tamanho da Entrada.

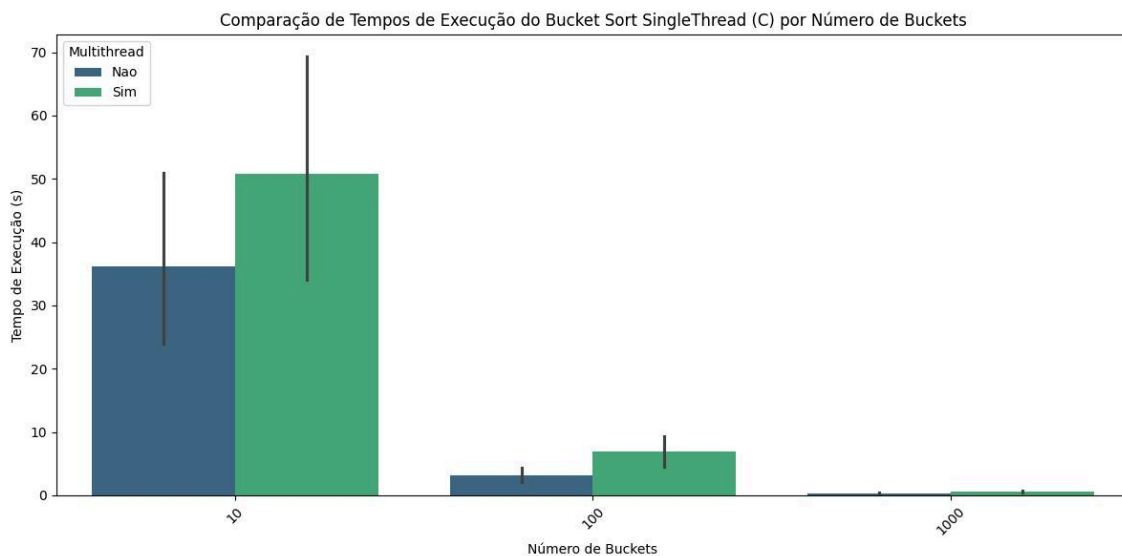


Figura 11. Bucket Sort *Single Thread* x Bucket Sort *Multi Thread* x Números de *Buckets*.

Na Figura 12, com um conjunto de números aleatórios, percebe-se que conforme o número de baldes aumentava, o tempo de execução diminuía, e novamente vemos o fato da paralelização não ser mais eficiente, tendo tempos por vezes superiores. Na Figura 11, com um conjunto de números parcialmente ordenados, podemos perceber uma certa irrelevância na alteração do número de baldes, uma vez que os tempos para cada conjunto de baldes, 10, 100 e 1000, se manteve bem próximo, o comportamento do *Multi Thread* também se manteve.

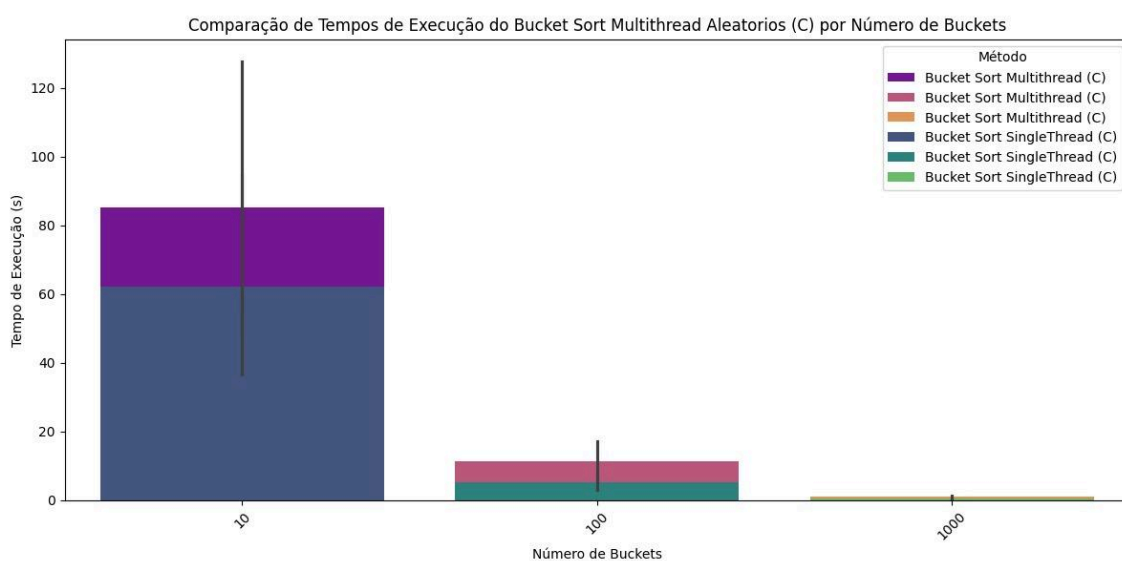


Figura 12. Bucket Sort *Single Thread* x *Multi Thread* x Números Aleatórios.

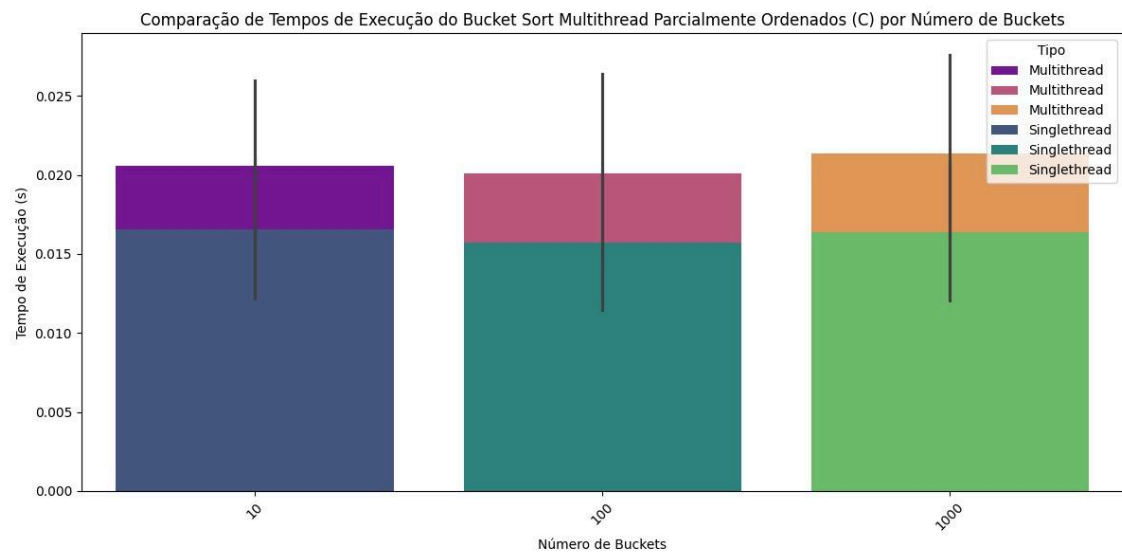


Figura 13. Bucket Sort *Single Thread* x Bucket Sort *Multi Thread* x Números Parcialmente Ordenados.

Na Figura 14, com um conjunto de números ordenados, é mostrado que independente do número de baldes, o tempo de execução tem a mesma média, com uma variação muito pequena, que em alguns momentos faz o menor número de baldes, 10, ser melhor que maior, mesmo com mais elementos. Ao fazer um paralelo com a Figura 13, vemos que o Bucket Sort para elementos já ordenados mostrou um desperdício de desempenho, elencando que somente o Insertion Sort já daria conta para esses caso, utilizando o paralelismo então, é mais notável esse desperdício, mesmo assim o tempo de execução ainda é irrisório humanamente falando, pois não chegou a nem 0.1 segundos em ambos os casos.

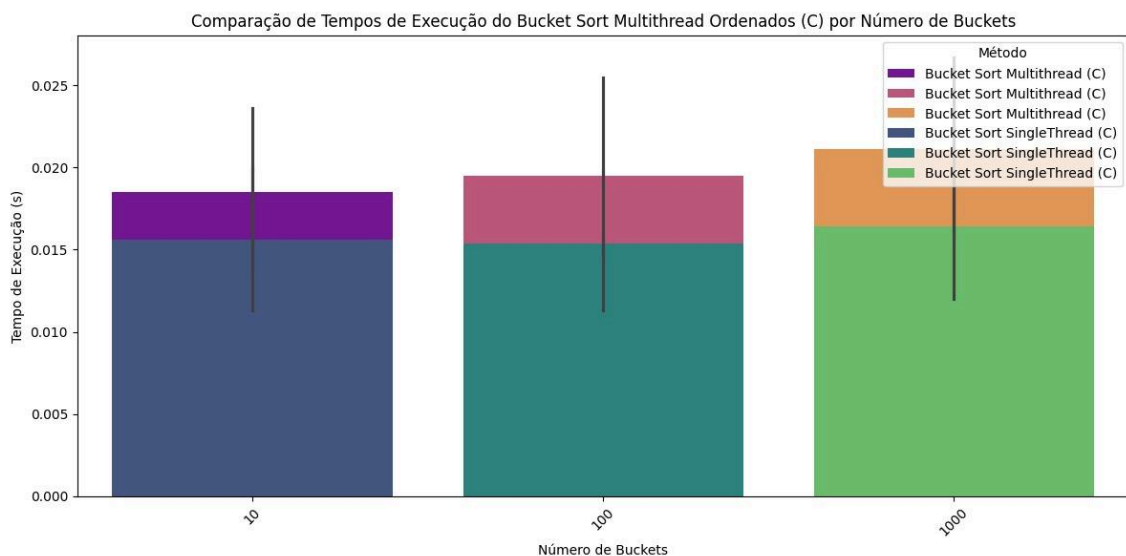


Figura 14. Bucket Sort *Single Thread* x *Multi Thread* x Números Ordenados.

Na Figura 15, com um conjunto de elementos decrescentes, volta-se a notar a diminuição do tempo de execução conforme o número de baldes aumenta, mostrando novamente a efetividade para esses casos, mesmo assim o tempo ainda é superior a 100 segundos para 10 baldes, um tempo considerável, que reduz para mais da metade com 100 baldes e 90% com 1000 baldes.

Nas figuras apresentadas, um certo padrão foi encontrado, a piora de desempenho com o uso do paralelismo, revelando a ineficácia desse método para o Bucket Sort.

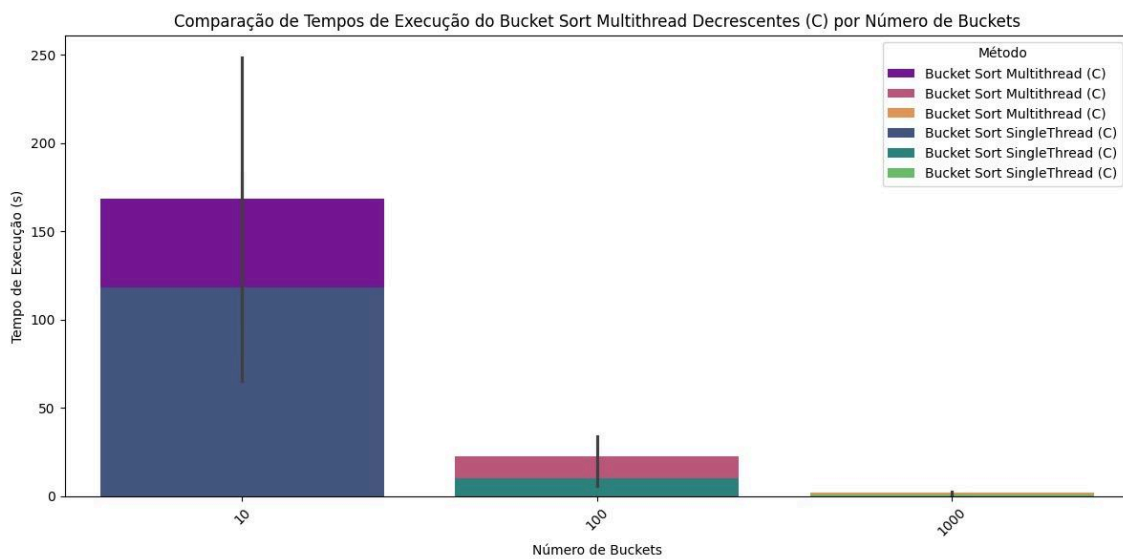


Figura 15. Bucket Sort *Single Thread* x *Multi Thread* x Números Decrescentes.

O Quadro 2 apresenta os tempos médios de execução do algoritmo Bucket Sort em C++ para diferentes tipos de entrada de dados (Aleatórios, Decrescentes, Ordenados e Parcialmente Ordenados), tanto em sua versão single-thread (Não) quanto *multi thread* (Sim).

Ao analisar os dados, observa-se que, na modalidade *single thread*, o Bucket Sort apresenta um desempenho mais eficiente para entradas "Ordenados" e "Parcialmente Ordenados", com tempos médios de 0.015 segundos. Isso se alinha com a expectativa de que o algoritmo se beneficia de uma distribuição inicial mais uniforme dos elementos nos baldes. Para entradas "Aleatórios" e "Decrescentes", os tempos médios aumentam para 22.675 segundos e 43.253 segundos, respectivamente, indicando que a organização inicial dos dados impacta a performance.

Surpreendentemente, ao introduzir o paralelismo (*multi thread*), o desempenho do Bucket Sort não melhora, e em alguns casos, piora. Para entradas "Aleatórios", o tempo médio aumenta para 32.627 segundos (contra 22.675 s em single-thread), e para "Decrescentes", sobe para 64.389 segundos (contra 43.253 s). Mesmo para os casos

"Ordenados" e "Parcialmente Ordenados", onde a diferença é marginal, os tempos *multi thread* são ligeiramente superiores (0.019 s e 0.021 s, respectivamente). Esse comportamento sugere que o overhead de sincronização e gerenciamento de threads pode estar anulando os potenciais benefícios do paralelismo para o Bucket Sort neste cenário, tornando a implementação single-thread mais eficiente para os tipos de entrada testados.

Multithread	Tipo de Entrada	Tempo de Execução Médio (s)
Não	Aleatórios	22.675
Não	Decrescentes	43.253
Não	Ordenados	0.015
Não	Parcialmente Ordenados	0.015
Sim	Aleatórios	32.627
Sim	Decrescentes	64.389
Sim	Ordenados	0.019
Sim	Parcialmente Ordenados	0.021

Quadro 2. Comparação entre os tipos de entrada para o BucketSort, *multi thread* e *single thread*.

4.2.3 Insertion Sort X Bucket Sort - 10, 100 e 1000 em C++

Nas Figuras 16 e 17, vemos uma comparação do método do Insertion Sort com Bucket Sort, utilizando Insertion Sort, em suas variações de número de baldes. Na Figura 14, revela-se que o tempo médio de execução do Insertion Sort, quando comparado ao Bucket Sort é 75% maior, o que mostra que utilizar o Bucket Sort junto do Insertion Sort é altamente eficaz para *Single Thread*. Na imagem 14 então, vemos a comparação do Insertion *Multi Thread*, com o Bucket e suas variações, ao fazer uma análise dessa imagem, vemos que a paralelização do Insertion trouxe resultados altamente expressivos, chegando bem próximo do Bucket Sort com 10 baldes, e uma média geral de menos de 50 segundo, isso revela que caso seja possível paralelizar o

método do Insertion Sort, é uma ótima aplicação e caso não seja possível, a melhor posição seria a junção do *Bucket* com o *Insertion*, pois assim se consegue resultados mais expressivos em termos de desempenho e tempo de execução.

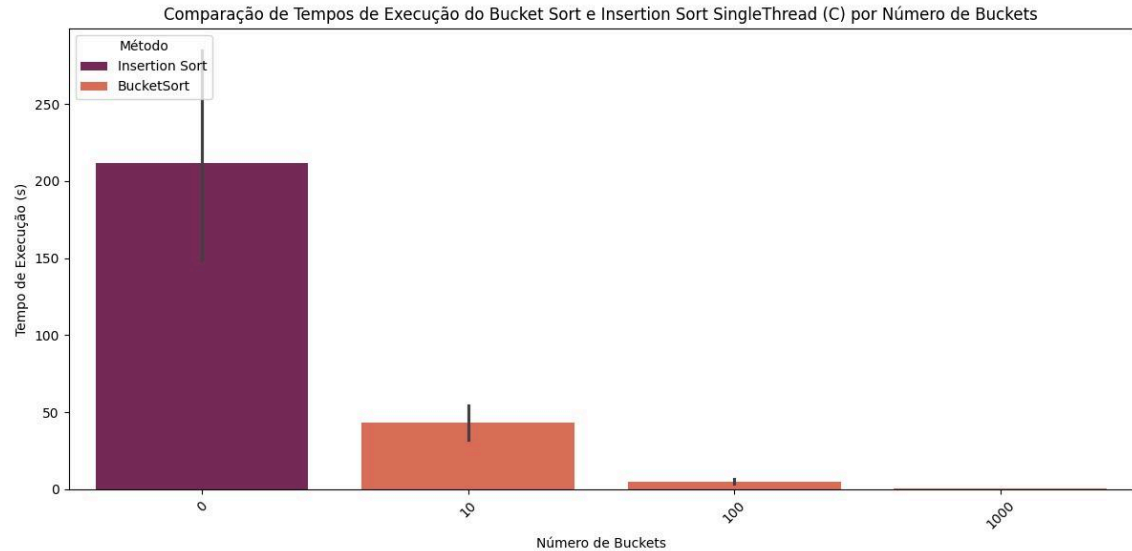


Figura 16. Insertion Sort *Single Thread* x Bucket Sort *Single Thread* x Número de Buckets.

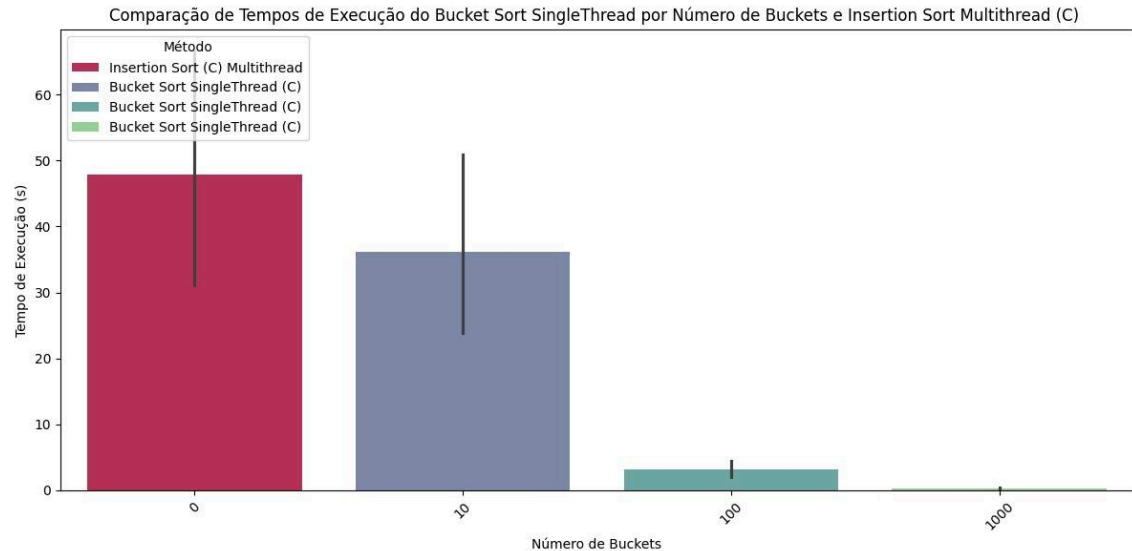


Figura 17. Insertion Sort *Multi Thread* x Bucket Sort *Single Thread* x Número de Buckets.

Na Figura 16, vemos um comparativo geral de todos os métodos analisados com relação aos tempos de execução, um detalhe relevante, como visto na Figura 15, o

Insertion Sort *Multi Thread* é até melhor que o Bucket Sort *Multi Thread* com 10 baldes, novamente mostrando a sua eficiência.

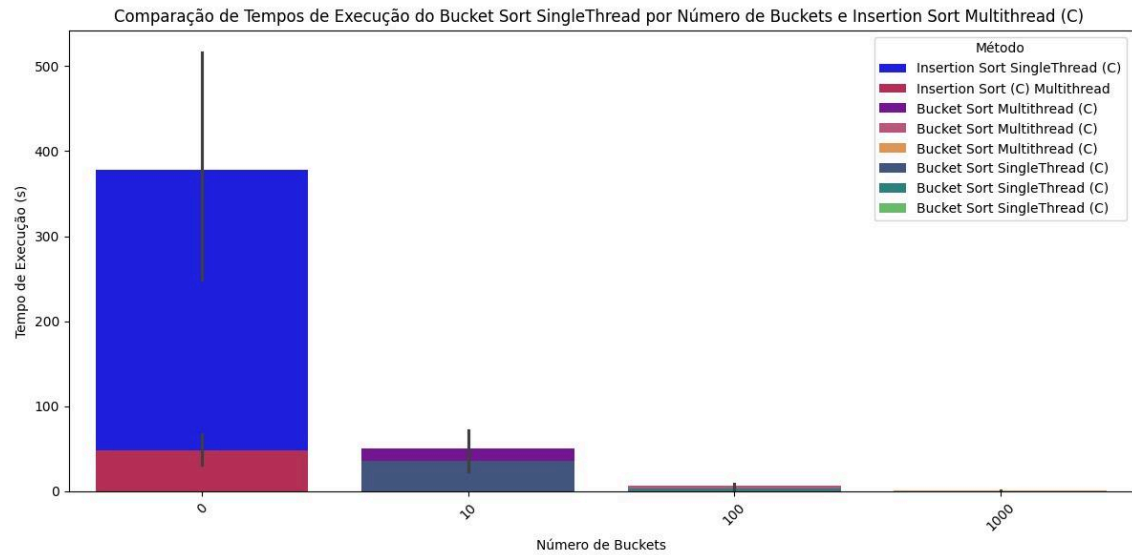


Figura 16. Insertion Sort *Single Thread* x Bucket Sort *Single Thread* x Insertion Sort *Multi Thread* x Bucket Sort *Multi Thread*.

O Quadro 3 apresenta uma comparação detalhada do desempenho dos algoritmos Bucket Sort e Insertion Sort com números brutos, também considerando variações de paralelismo e o número de baldes para o Bucket Sort. Observa-se que, no contexto do Bucket Sort sem paralelismo (*Multithread*: Não), o tempo de execução médio diminui significativamente à medida que o número de baldes aumenta. Com 10 baldes, a média é de 36.16 segundos, caindo para 3.16 segundos com 100 baldes e atingindo 0.27 segundos com 1000 baldes. O desvio padrão também reflete essa tendência, indicando menor variabilidade em execuções com mais baldes.

No entanto, ao introduzir o paralelismo (*Multithread*: Sim) no Bucket Sort, os resultados demonstram um comportamento inesperado. Para 10 baldes, o tempo médio de execução aumenta para 50.79 segundos (contra 36.16 s sem paralelismo), e para 100 baldes, sobe para 6.86 segundos (contra 3.16 s). Embora com 1000 baldes o tempo médio seja de 0.59 segundos, ainda é superior ao seu equivalente *single-thread* (0.27 s). Isso sugere que o *overhead* de sincronização das *threads* pode ter impactado negativamente o desempenho do Bucket Sort paralelizado neste cenário.

Em contraste, o Insertion Sort exibe uma performance distinta. Na versão *single thread* (*Multithread*: Não), o tempo médio de execução é de 377.49 segundos, com um desvio padrão de 1386.19. Contudo, a aplicação de paralelismo (*Multithread*: Sim) no Insertion Sort resulta em uma drástica redução no tempo médio, caindo para 47.93 segundos, com um desvio padrão de 178.82. Essa melhoria substancial demonstra a eficácia da paralelização para o Insertion Sort, diferentemente do que foi observado no Bucket Sort.

Método	Multithread	Buckets	Tempo de execução Desvio Padrão (s)	Tempo de execução Média (s)
Bucket Sort	Não	10	137.35	36.16
Bucket Sort	Não	100	12.21	3.16
Bucket Sort	Não	1000	0.99	0.27
Bucket Sort	Sim	10	192.45	50.79
Bucket Sort	Sim	100	25.41	6.86
Bucket Sort	Sim	1000	2.13	0.59
Insertion Sort	Não	0	1386.19	377.49
Insertion Sort	Sim	0	178.82	47.93

Quadro 3. Comparação entre o Bucket Sort e Insertion Sort, *multi thread*, *single thread* com as médias e desvios padrões.

4.2.4 C++ versus Python

Nas Figuras 17 e 18 são mostrados os gráficos comparativos das médias do tempo de execução dos métodos Insertion Sort e Bucket Sort, respectivamente, *Single Thread*. Na Figura 17, nota-se que conforme o número de elementos vai crescendo, o tempo de execução em Python vai cada vez aumentando mais quando comparado com C++, a partir de quinze mil elementos, já se torna altamente visível, quando se chega a 50 mil, ocorre um abismo de desempenho, onde em Python o tempo de execução é superior a 60 segundos, enquanto que em C++ não ultrapassa 10 segundos, executando em menos tempo inclusive. Na Figura 18, ao olhar o Bucket Sort, vemos novamente uma grande diferença de desempenho entre as linguagens, uma vez que o tempo médio de até 50 mil elementos em Python é cerca de 1.2 segundos para 10 baldes, enquanto em C++ é menor que 0.1 segundo, ao adotar que em C++ foi 0.1 segundo, há uma diferença de desempenho de aproximadamente 91% para 10 baldes. Conforme o número de baldes aumenta essa diferença, logicamente, reduz, porém ainda é bastante relevante em um contexto avaliando desempenho.

Python é valorizada por sua facilidade de uso e vasta biblioteca, sendo ótima para prototipagem e software de pequena escala. Contudo, seu desempenho é limitado

em sistemas maiores devido ao seu caráter interpretado. Para compensar, bibliotecas como NumPy e Cython melhoram a performance através de operações de baixo nível e integração com C/C++ [5]. Em contexto de algoritmos que exigem um poder computacional, Python pode não ser a linguagem mais interessante, mesmo que muitas vezes ela seja escolhida para realização de tarefas mais comuns, devido a sua ampla gama de bibliotecas voláteis e versáteis, poucas realmente tem a necessidade de desempenho como o Insertion Sort e Bucket Sort necessitam, o que torna esse linguagem pouco eficiente.

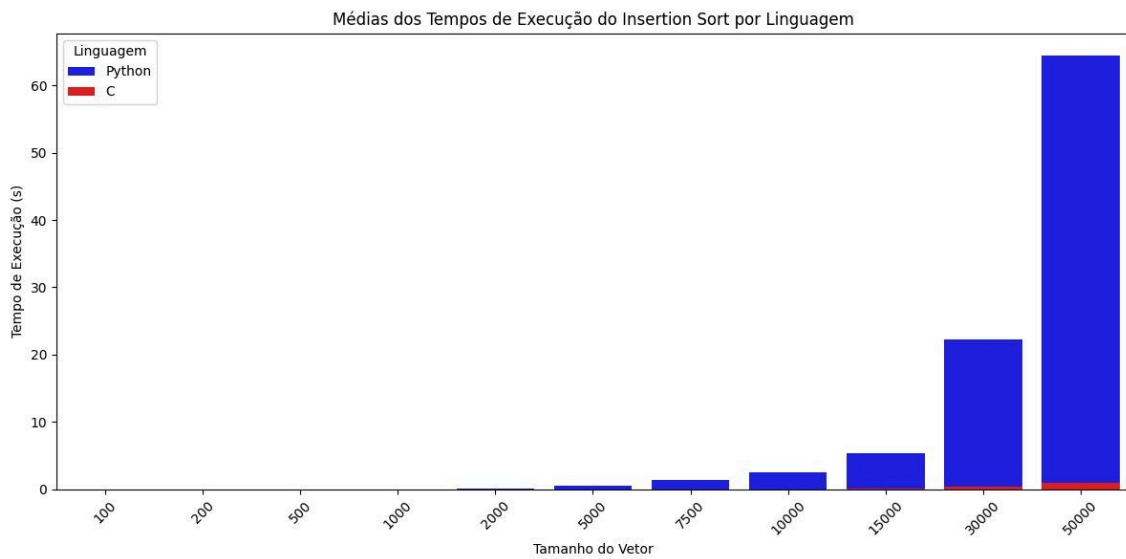


Figura 17. Insertion Sort em Python x Insertion Sort em C++.

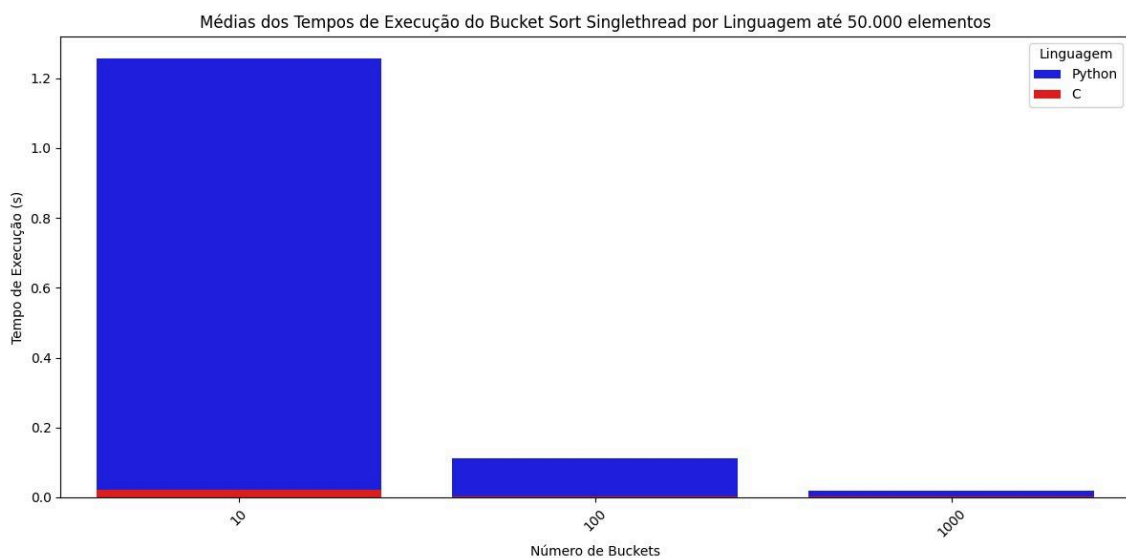


Figura 18. Bucket Sort em Python x Bucket Sort em C++

Nos experimentos realizados, inicialmente não houve limitação de número de elementos em Python, porém após um período de dois dias onde não houve uma

evolução dos conjuntos acima de 1 milhão de elementos, ficando travado durante a execução, optou-se por limitar a 50 mil elementos, pois dessa forma já seria notável a grande diferença de desempenho e seria possível avaliar todos os casos. Como o objetivo deste artigo foca na comparação dos métodos Insertion Sort e Bucket Sort com variações e não em uma comparação de Python com C++ na aplicação desses métodos, alguns testes ainda devem ser realizados para uma análise mais profunda e concreta, porém essa breve análise já revela um caminho provável dos resultados que podem ser encontrados com o estudo mais profundo.

Devido a inviabilidade de rodar os métodos na linguagem Python com todos os conjuntos de elementos para execução do trabalho, acaba ficando em aberto essa comparação, mas, novamente, já se nota um desfecho para uma avaliação maior.

5. Conclusão

A análise comparativa entre a simplicidade quadrática do Insertion Sort e a eficiência distributiva do Bucket Sort, potencializada pela variação no número de baldes e pela aplicação de paralelismo, chega agora ao seu momento de síntese. Os resultados experimentais obtidos revelam pontos importantes e interessantes com relação ao uso do método do Insertion Sort e do Bucket Sort.

A priori, é perceptível a superioridade do Bucket Sort com relação ao Insertion Sort, esse comportamento já era esperado devido ao conhecimento prévio do custo assintótico de ambos os métodos, porém os experimentos mostraram na prática que esse comportamento também é refletido na prática, além da teoria. Detalhes inesperados também foram percebidos e ponderados, como o fato do Bucket Sort *Multi Thread* apresentar um desempenho pior quando comparado ao *Single Thread*, um comportamento que vai contra a teoria de que ao se paralelizar algo, ganha-se desempenho. Notou-se também a alta falta de eficiência do Insertion Sort *Single Thread* conforme o número de elementos aumenta, e isso foi visto durante a sua execução com o conjunto de números decrescentes, seu pior caso, onde para dois milhões de elementos ele chegou a uma execução de mais de dez mil segundos, cerca de 2 horas e 47 minutos. A grande surpresa se revelou com o uso do paralelismo no Insertion Sort, que teve um aumento de desempenho considerável para todos os conjuntos de dados

A posteriori, como um experimento complementar deste estudo, a análise comparativa das linguagens Python e C++, que revelou alguns detalhes interessantes com relação a desempenho, uma vez que a diferença de tipo de linguagem realmente pode afetar os resultados obtidos. Para a análise comparativa complementar esperada de ambos os métodos (Insertion Sort e Bucket Sort) em ambas as linguagens, houve limitações com relação à capacidade de atingir as execuções, uma vez que o gasto computacional de Python se tornou um grande desafio a ser superado, conseguindo apenas uma micro visão de um cenário limitado. Tal limitação impediu que os testes fossem executados em larga escala na linguagem interpretada, comprometendo a equidade da comparação. Ainda assim, foi possível observar que, mesmo com algoritmos relativamente simples, a eficiência de C++ — linguagem compilada — se destacou, reforçando a importância de considerar o ambiente de implementação como fator crítico em avaliações de desempenho.

Referencias

- [1] GEEKSFORGEEEKS. Insertion Sort Algorithm (2013). Disponível em: <<https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/>>. Acesso em: 14 jul. 2025.
- [2] GEEKSFORGEEEKS. Bucket Sort Data Structures and Algorithms Tutorials. Disponível em: <<https://www.geeksforgeeks.org/dsa/bucket-sort-2/>>. Acesso em: 14 jul. 2025.
- [3] REIS, F. DOS. O que é Notação Big O em programação e análise de algoritmos - Bóson Treinamentos em Ciência e Tecnologia. Disponível em: <<https://www.bosontreinamentos.com.br/estruturas-de-dados/o-que-e-notacao-big-o-em-programacao-e-analise-de-algoritmos/>>. Acesso em: 14 jul. 2025.
- [4] Paralelismo de dados (execução de simultaneidade). [s.l: s.n.]. Disponível em: <<https://www.inf.ufsc.br/~bosco.sobral/ensino/ine5645/Paralelismo%20de%20Dados%20Paralelismo%20de%20Tarefas.pdf>>. Acesso em: 14 jul. 2025.
- [5] Uma comparação empírica em velocidade de processamento entre C++, Go, Rust, Python e JavaScript. Disponível em: <<https://lume.ufrgs.br/handle/10183/272878>>. Acesso em: 17 jul. 2025.